```
// g++ -O2 -std=c++17 b3_verify_exact.cpp -lgmp -o b3_verify_exact
// ./b3_verify_exact > b3_verification_output.txt
// Exact numerical certificate for Lemma 2.10, equations (2.84)-(2.102).
// All verification arithmetic uses GMP rationals; no tolerance is used.
// The functional estimates and monotonicity arguments are supplied by
// the paper. This program certifies their rational endpoint constants.

#include <gmp.h>
#include <chrono>
#include <iostream>
#include <stdexcept>
#include <string>
#include <utility>

class Q
{
    mpq_t value;

  public:
    Q(long numerator = 0, unsigned long denominator = 1)
    {
        if (denominator == 0)
            throw std::invalid_argument("Zero denominator");
        mpq_init(value);
        mpq_set_si(value, numerator, denominator);
        mpq_canonicalize(value);
    }

    explicit Q(const std::string &text)
    {
        mpq_init(value);
        if (mpq_set_str(value, text.c_str(), 10) != 0)
        {
            mpq_clear(value);
            throw std::invalid_argument("Invalid rational literal");
        }
        mpq_canonicalize(value);
    }

    Q(const Q &other)
    {
        mpq_init(value);
        mpq_set(value, other.value);
    }

    Q(Q &&other) noexcept
    {
        mpq_init(value);
        mpq_swap(value, other.value);
    }

    Q &operator=(Q other) noexcept
    {
        mpq_swap(value, other.value);
        return *this;
    }

    ~Q()
    {
        mpq_clear(value);
    }

    friend Q operator+(const Q &a, const Q &b)
    {
        Q result;
        mpq_add(result.value, a.value, b.value);
        return result;
```

```
    }

    friend Q operator-(const Q &a, const Q &b)
    {
        Q result;
        mpq_sub(result.value, a.value, b.value);
        return result;
    }

    friend Q operator*(const Q &a, const Q &b)
    {
        Q result;
        mpq_mul(result.value, a.value, b.value);
        return result;
    }

    friend Q operator/(const Q &a, const Q &b)
    {
        if (mpq_sgn(b.value) == 0)
            throw std::invalid_argument("Division by zero");
        Q result;
        mpq_div(result.value, a.value, b.value);
        return result;
    }

    friend bool operator<(const Q &a, const Q &b)
    {
        return mpq_cmp(a.value, b.value) < 0;
    }

    friend bool operator==(const Q &a, const Q &b)
    {
        return mpq_cmp(a.value, b.value) == 0;
    }

    friend std::ostream &operator<<(std::ostream &out, const Q &a)
    {
        char *buffer = mpq_get_str(nullptr, 10, a.value);
        std::string text(buffer);
        void (*release)(void *, size_t);
        mp_get_memory_functions(nullptr, nullptr, &release);
        release(buffer, text.size() + 1);
        return out << text;
    }
};

Q power(Q base, unsigned exponent)
{
    Q result(1);
    while (exponent)
    {
        if (exponent & 1U)
            result = result * base;
        base = base * base;
        exponent >>= 1;
    }
    return result;
}

Q positive_exp_sum(const Q &x, unsigned degree)
{
    if (x < Q(0))
        throw std::invalid_argument("Taylor argument must be nonnegative");
    Q sum(1), term(1);
    for (unsigned k = 1; k <= degree; ++k)
    {
        term = term * x / Q(k);
```

```
        sum = sum + term;
    }
    return sum;
}

// Alternating-series bounds for atan(x), with 0 < x < 1.
std::pair<Q, Q> atan_bounds(const Q &x, unsigned terms)
{
    if (!(Q(0) < x && x < Q(1)) || terms == 0)
        throw std::invalid_argument("Invalid arctangent parameters");
    Q sum(0), xpower = x;
    for (unsigned k = 0; k < terms; ++k)
    {
        Q term = xpower / Q(2 * k + 1);
        sum = (k & 1U) ? sum - term : sum + term;
        xpower = xpower * x * x;
    }
    Q next = xpower / Q(2 * terms + 1);
    if (terms & 1U)
        return {sum - next, sum};
    return {sum, sum + next};
}

int checks = 0, failures = 0;

void check(bool condition, const char *equation, const char *statement)
{
    ++checks;
    if (!condition)
        ++failures;
    std::cout << (condition ? "PASS " : "FAIL ") << checks << ' ' << equation << ' '
              << statement << '\n';
}

int main()
{
    try
    {
        const auto start = std::chrono::steady_clock::now();
        const Q S(330000), N(1000000), root_lower(570), root_upper(575);
        const Q c_lower(64, 25), theta(2, 25);
        const Q ten24 = power(Q(10), 24);

        // Machin's identity: pi = 16 atan(1/5) - 4 atan(1/239).
        const auto a = atan_bounds(Q(1, 5), 12);
        const auto b = atan_bounds(Q(1, 239), 4);
        const Q pi_lower = Q(16) * a.first - Q(4) * b.second;
        const Q pi_upper = Q(16) * a.second - Q(4) * b.first;
        check(Q(157, 50) < pi_lower, "[2.1]", "157/50 < pi");
        check(pi_upper < Q(22, 7), "[2.1]", "pi < 22/7");
        check(Q(41, 25) < power(Q(157, 50), 2) / Q(6), "[2.1]", "41/25 < (157/50)^2/6");
        check(power(Q(22, 7), 2) / Q(6) < Q(33, 20), "[2.1]", "(22/7)^2/6 < 33/20");
        check(power(c_lower, 2) < Q(4) * Q(41, 25), "[2.1]", "(64/25)^2 < 4*(41/25)");
        check(Q(4) * Q(33, 20) < power(Q(18, 7), 2), "[2.1]", "4*(33/20) < (18/7)^2");

        // All s-dependent endpoint checks below use s >= S.
        check(S < N / Q(3) - Q(5), "[2.84]", "330000 < 10^6/3 - 5");
        check(Q(15) < S, "[2.84]", "3*s + 15 <= 4*s for s >= 330000");
        check(power(root_lower, 2) < S, "[2.84]", "570^2 < 330000");
        check(S < power(root_upper, 2), "[2.84]", "330000 < 575^2");
        check(Q(1000) * Q(1000) == N, "[2.84]", "sqrt(10^6) = 1000");
        check(Q(4) + Q(1, 24) < S / Q(10), "[2.86]", "s - 4 - 1/24 > 9*s/10");
        check(Q(100) < Q(9, 10) * S, "[2.86]", "9*s/10 > 100");
        check(Q(0) < Q(2000) - Q(1) - Q(1, 24), "[2.86]",
              "t - 2 - 1/24 > s at the smallest m");
        check(Q(0) < Q(2000) + Q(1) - Q(4), "[2.87]",
              "smallest denominator argument exceeds numerator arguments");
```

```
check(Q(20) < S, "[2.87]", "3/(s-5) <= 4/s");
check(power(Q(1) - Q(1) / S, 2) * (Q(1) + Q(2) / S) < Q(1), "[2.87]",
      "1/sqrt(1+2/s) >= 1-1/s");
check(Q(127, 100) < Q(32, 25) - Q(32, 25) / S - Q(4) / root_lower, "[2.87]",
      "32/25 - 32/(25*330000) - 4/570 > 127/100");
check(Q(16) * power(S, 3) < Q(25) * power(S - Q(5), 3), "[2.88]",
      "4*(s-5)^(-3/2) < 5*s^(-3/2)");
check(Q(8, 5) < power(Q(127, 100), 2) - Q(5) / root_lower, "[2.88]",
      "(127/100)^2 - 5/570 > 8/5");
check(power(Q(9, 7), 2) * Q(10, 9) < power(Q(34, 25), 2), "[2.90]",
      "(9/7)*sqrt(10/9) < 34/25");
check(Q(127, 100) < Q(32, 25) - Q(32, 25) / S - Q(5, 3) / root_lower, "[2.90]",
      "numerator lower endpoint exceeds 127/100");
check(Q(2) + Q(10) / S < power(Q(283, 200), 2), "[2.91]",
      "sqrt(2*s+10) < (283/200)*sqrt(s)");
check(Q(9, 10) < Q(256, 283) - Q(3, 2) / root_lower, "[2.91]",
      "256/283 - 3/(2*570) > 9/10");
check(Q(9, 7) < Q(2) * Q(127, 100), "[2.92]",
      "-127*r/100 + 9/7 < 0 for r >= 2");
check(Q(0) < Q(2) * Q(34, 25) - Q(9, 10), "[2.92]",
      "34*r/25 - 9/10 > 0 for r >= 2");
check(Q(2) + Q(16) / root_lower < Q(3), "[2.93]", "2 + 16/570 < 3");
check(Q(1) + Q(1) / power(Q(2), 2) == Q(5, 4), "[2.93]",
      "1 + 1/r^2 <= 5/4 for r >= 2");
check(Q(15, 4) / root_lower < Q(1, 150), "[2.93]", "15/(4*570) < 1/150");

const Q x = Q(127, 100) * (Q(2) - Q(5, 1000));
check(Q(0) < x, "[2.95]", "exponential lower argument is positive");
check(x == Q(50673, 20000), "[2.95]", "lower argument = 50673/20000");
check(Q(25, 2) < positive_exp_sum(x, 10), "[2.95]", "S_10(50673/20000) > 25/2");
check(Q(0) < theta && theta < Q(1), "[2.95]", "0 < theta=2/25 < 1");

// Compute sum_{r>=3} r^j theta^(r-2) independently from
// G=1/(1-theta), using the operator theta*d/dtheta.
const Q d = Q(1) - theta;
const Q g1 = Q(1) / power(d, 2);
const Q g2 = Q(2) / power(d, 3);
const Q g3 = Q(6) / power(d, 4);
const Q raw1 = theta * g1;
const Q raw2 = raw1 + power(theta, 2) * g2;
const Q raw3 = raw1 + Q(3) * power(theta, 2) * g2 + power(theta, 3) * g3;
const Q m1 = (raw1 - theta - Q(2) * power(theta, 2)) / power(theta, 2);
const Q m2 = (raw2 - theta - Q(4) * power(theta, 2)) / power(theta, 2);
const Q m3 = (raw3 - theta - Q(8) * power(theta, 2)) / power(theta, 2);
check(m1 == theta * (Q(3) - Q(2) * theta) / power(d, 2), "[2.97]",
      "first geometric moment formula");
check(m2 ==
          theta * (Q(9) - Q(11) * theta + Q(4) * power(theta, 2)) / power(d, 3),
      "[2.97]", "second geometric moment formula");
check(m3 == theta *
                (Q(27) - Q(44) * theta + Q(31) * power(theta, 2) -
                 Q(8) * power(theta, 3)) /
                power(d, 4),
      "[2.97]", "third geometric moment formula");
std::cout << "VALUE [2.97] moments=" << m1 << ',' << m2 << ',' << m3 << '\n';

const Q budget = (power(Q(34, 25), 2) + Q(1, 150)) * m3 / Q(2) -
                 Q(153, 125) * m2 + Q(81, 200) * m1;
check(budget == Q(std::string("3228244387/2098807500")), "[2.96]",
      "curvature budget equals 3228244387/2098807500");
check(budget < Q(31, 20), "[2.96]", "curvature budget < 31/20");
std::cout << "VALUE [2.96] curvature_budget=" << budget << '\n';
check(m1 / Q(2) == Q(71, 529), "[2.95]", "later-term mass = 71/529");

// The following monotonic endpoint reductions are those in the paper.
const Q z = Q(91, 25) / root_lower;
```

```
        check(Q(0) < z && z < Q(1, 2), "[2.99]", "0 < 91/(25*570) < 1/2");
        check(Q(1) / (Q(1) - z) < Q(2), "[2.99]", "exp(z) <= 1/(1-z) < 2");
        check(Q(2) + Q(71, 529) < Q(3), "[2.99]", "smooth bulk mass < 3");
        check(Q(1600) * S * root_upper < ten24, "[2.99]",
              "1600*330000*575 < 10^24; used with S_50(57)");
        check(Q(3) < root_lower, "[2.99]", "s^(3/2)*exp(-sqrt(s)) is decreasing");
        check(Q(192) == Q(8) * Q(2) * Q(12), "[2.100]",
              "numerator replacement coefficient = 192");
        check(c_lower * Q(1, 2) - c_lower < Q(-5, 4), "[2.100]",
              "constant coefficient in bulk exponent is below -5/4");
        check(c_lower * Q(9, 2) < Q(27, 2), "[2.100]",
              "1/s coefficient in bulk exponent is below 27/2");
        check(Q(54) < S, "[2.100]", "-5/4 + 27/(2*s) < -1");
        check(Q(768) == Q(4) * Q(192), "[2.100]", "bulk numerator coefficient = 768");
        check(Q(4800) == Q(2) * Q(800) * Q(3), "[2.100]",
              "bulk denominator coefficient = 4800");
        check(Q(768) + Q(4800) / root_lower < Q(6000), "[2.100]",
              "bulk value error coefficient < 6000");
        check(Q(24000) == Q(4) * Q(6000), "[2.100]",
              "second-difference error coefficient = 24000");

        check(Q(6001) < N / Q(150) - Q(1, 10), "[2.101]", "6001 < 10^6/150 - 1/10");
        check(Q(0) < Q(1, 150) - Q(3, 1000), "[2.101]",
              "derivative of n/150 - 1/10 - 6*sqrt(n) - 1 is positive");
        check(Q(9, 10) + Q(1, 50) == Q(23, 25), "[2.101]", "9/10 + 1/50 = 23/25");
        const Q aroot(9999, 10000), broot(9592, 10000);
        check(power(aroot, 2) < Q(1) - Q(4) / S, "[2.102]",
              "sqrt(s-4) > (9999/10000)*sqrt(s)");
        check(Q(23, 25) < power(broot, 2), "[2.102]",
              "sqrt(23*s/25) < (9592/10000)*sqrt(s)");
        check(Q(1, 10) < c_lower * (aroot - broot), "[2.102]",
              "(64/25)*(9999-9592)/10000 > 1/10");
        check(Q(96) == Q(4) * Q(2) * Q(12), "[2.102]",
              "nonbulk value coefficient = 96");
        check(Q(384) == Q(4) * Q(96), "[2.102]",
              "nonbulk second-difference coefficient = 384");
        check(Q(24000) + Q(384) < Q(25000), "[2.102]", "24000 + 384 < 25000");
        check(Q(60) < root_lower, "[2.102]", "s^3*exp(-sqrt(s)/10) is decreasing");
        check(Q(57) == root_lower / Q(10), "[2.102]", "570/10 = 57");
        check(ten24 < positive_exp_sum(Q(57), 50), "[2.102]", "S_50(57) > 10^24");
        check(Q(25000) * power(S, 3) / ten24 < Q(1, 1000), "[2.102]",
              "25000*330000^3/10^24 < 1/1000");
        std::cout << "VALUE [2.102] endpoint_error_upper="
                  << Q(25000) * power(S, 3) / ten24 << '\n';
        const Q final_margin = Q(8, 5) - Q(31, 20) - Q(1, 1000);
        check(final_margin == Q(49, 1000), "[2.102]", "8/5 - 31/20 - 1/1000 = 49/1000");
        check(Q(0) < final_margin, "[2.102]", "final curvature margin > 0");
        std::cout << "VALUE [2.102] final_margin=" << final_margin << '\n';

        const auto elapsed = std::chrono::duration_cast<std::chrono::microseconds>(
                                 std::chrono::steady_clock::now() - start)
                                 .count();
        std::cout << (failures ? "COMPLETE_FAIL" : "COMPLETE_PASS")
                  << " checks=" << checks << " failed=" << failures
                  << " arithmetic=GMP_exact_rational elapsed_us=" << elapsed << '\n';
        return failures ? 1 : 0;
    }
    catch (const std::exception &error)
    {
        std::cerr << "ERROR " << error.what() << '\n';
        return 2;
    }
}
```